ATR Pipeline — Work Summary

Single-tool (BM25 MuSiQue) agentic tool-use training contract — data-quality hardening, route holdout, and passage naturalization

Prepared by the engineering agent · verified locally (Windows / CPU)

1. Objective

Build and validate a data pipeline for training an agent to solve multi-hop questions by chaining BM25 *search* tool calls. The current SFT model sits at a **20% TASK SUCCESS** baseline against the internal dev set; the immediate target is to lift this before GRPO. The work here focuses on **training-data quality**: removing questions that can be answered without chaining, holding out structural shapes for honest evaluation, and making synthetic passages look more natural without changing the underlying facts.

2. What was changed (three parts)

Part 1 — Shortcut (disconnection) filter

A multi-hop question is a *disconnected shortcut* when a single, un-chained BM25 search can already surface the gold answer — the exact lazy move a model makes instead of reasoning. We now detect this with the **same *search* tool the agent uses**: fire one search with the full question text and check whether the returned passages already contain the gold value. Such questions are rejected at generation time. The filter is off by default for fast/offline work and is observable via a `_SHORTCUT_STATS` counter.
Measured on the wired training path (200 tasks, seeds 0–199), the shortcut filter is observably live: of **150** candidate multi-hop chains checked, **24 were rejected** (≈**16.0%**) as shortcut-solvable — a real, non-trivial leak rather than dead code. (An earlier 500-task sample measured ≈14.5%.)

How it works (function flow)

- **gen_musique()** builds the final `prompt` for each candidate chain; if `filter_shortcuts=True` it increments `_SHORTCUT_STATS["checked"]` then calls `_is_shortcut_solvable(w, prompt, gold)`.
- **_is_shortcut_solvable()**: `get_registry("builtin")` → `reg.call(w, "search", {"query": prompt, "top_k": 3})` — fires the **real BM25 search tool** with the full question text, returning top-3 passages. If 0 results → not a shortcut. It computes `want = _norm_find(gold)` (the normalized expected answer).
- **Substring check**: if the normalized answer appears in any of the 3 returned passages' normalized text → returns True (shortcut).
- **Back in gen_musique**: on True it increments `_SHORTCUT_STATS["rejected"]` and `continue`s (skips this chain, tries the next route). If no chain survives it returns None and the caller falls back to a no_tool task.

Part 2 — Train / dev route-shape holdout

A **shape** is the *fixed skeleton*: a comma-separated sequence of relations that defines how many hops and which general relations to chain (e.g. org_city → city_country means “find the org's city, then that city's country”). It is a **label only** — it does not fix any specific content. Underneath a shape, the concrete entities are free to vary: any person, company, city or country from the world's pools can fill each slot, so a single shape generates many distinct trajectories with different questions and answers. The *only* constraint is that each relation fixes the **kind** of entity per slot (a city slot takes any city, a person slot takes any person) — the **instances** are what vary.
Example (real questions, same 3-hop dev shape work_person → person_org → org_city): “who is the author whose company is headquartered in Meridian City?”, “which author writes for the company based in Nyrmont?”, “name the author at the firm located in Lacerta”. Same skeleton, different people/companies/cities, different answers each time.

Hops	Train shapes	Dev-only (held-out) shapes
2-hop	3 shapes	1 shape: org_city → city_country
3-hop	3 shapes	1 shape: work_person → person_org → org_city
4-hop	2 shapes	1 shape: work_person → person_org → org_city → city_country

Totals: **train = 8 shapes** (3 + 3 + 2), **dev = 3 held-out shapes** (1 + 1 + 1). Dev evaluation only ever sees held-out shapes, so success on dev is a genuine generalization signal. A test in `test_shortcut_filter.py` proves the train pool never emits a dev-only shape and the dev set uses exactly the held-out shapes.

How it works (function flow)

- Two module-level dicts in `generator.py`: `_ROUTES_TRAIN` and `_ROUTES_DEV_ONLY`, each keyed by hop count `{2: [...], 3: [...], 4: [...]}`. Each entry is a list of step-key sequences; each sequence is one shape.
- **Selection inside gen_musique()**: `pool = (_ROUTES_TRAIN if route_pool == "train" else _ROUTES_DEV_ONLY)[hops]`, then `rng.shuffle(routes)` and it iterates the shapes in random order, returning the first chain that resolves and passes the shortcut filter.
- **Train path**: `generate()` → `gen_*_hop(..., route_pool="train")` (default) → only train shapes.
- **Dev path**: `dev_set()` calls `fn(..., route_pool="dev")` → only dev-only shapes.
- **Verification**: the test rebuilds the shape a task used from its `oracle_plan` (mapping each plan query's trailing keyword back to a relation via `_kw_to_step()`) and asserts dev = the held-out shapes and train never matches a dev shape.

Dataset composition (what the whole dataset contains)

The dataset is **not** only country/city chains. It is a controlled mix of five families (verified: musique_2hop, musique_3hop, musique_4hop, no_tool, unanswerable):

- **musique_2hop / 3hop / 4hop** — the chained questions across 8 train + 3 dev shapes (person / company / city / country + attributes).
- **no_tool** — a question answerable from one passage alone; correct behavior is zero tool calls. A 304-item bank covering gold kinds all_of, any_of, numeric, text.
- **unanswerable** — the fact genuinely does not exist in the corpus (e.g. fake products like “Zephyr Vale”); the model must abstain, not hallucinate.

Part 3 — LLM passage naturalization (facts untouched)

Synthetic passages currently use fixed per-kind templates (e.g. every country passage has the same three sentences). Part A rewrites them into varied, Wikipedia-like prose using an LLM — **offline only**, never inside training or evaluation. Two hard invariants are enforced by post-check: (1) every fact the original template surfaced must still appear in the new text (otherwise retry, fall back to the original after a cap); (2) naturalization only ever replaces `doc.text` — it never touches the entity `attrs` dict, so gold answers and every verifier are byte-identical. Usage: `scripts/70_naturalize_passages.sh` mints a seed-keyed cache, then `build_world(seed, text_loader=...)` loads it opt-in.

How it works (function flow)

- **naturalize_passages(world, llm_client)** — entry point. Loops every `world.documents`, calls `naturalize_passage` per doc, writes back only `d["text"]`, returns a stats dict (docs / naturalized / fell_back).
- **naturalize_passage(passage, llm_client)** — retry loop (max 3): `_build_prompt(passage)` builds the rewrite instruction (with FACTS json + the current templated passage); `_call_llm(client, prompt)` gets the rewrite (client is a callable or an object with `.complete(prompt)`); `_facts_present()` is the safety post-check. If all facts survived, accept and return.
- **Fact-presence check**: `_surfaced_facts()` decides which facts must survive (those that already appear in the original text, via `_format_for_check()`, plus the title); `_text_presence()` does a formatting-robust substring match (strips non-alphanumerics so 1,097,000 == 1097000).
- **Fallback**: if all 3 retries fail, ship the original templated text and mark `naturalized=False` — never ships broken prose.
- **Scoring isolation**: `out = dict(passage)` then only `out["text"]` changes; the `facts`/`attrs` dicts (the sole source of gold answers) are untouched.

- **Opt-in wiring**: `build_world(seed, text_loader=None)` — when a loader is given it calls `text_loader(seed, d["doc_id"])` and, if a string returns, sets `d["text"]` and marks `naturalized=True`. The offline script writes `{"seed:doc_id": text}` to a cache; `load_naturalized_loader(path)` reads it back into a loader.

Seed & cache — the key idea

- **Seed** = an integer that makes world generation deterministic: `build_world(seed)` always yields the same entities and passages. Training generation uses seeds 0–500k; dev uses 900k+.
- **Range** = which worlds need naturalized passages. The cache is keyed by `seed:doc_id`, and `build_world` only substitutes naturalized prose for seeds present in the cache. So a seed range is required to tell the script which worlds to preprocess; a seed not in the cache simply gets templated prose (a clean per-seed fallback).
- **Full range (one-time preprocessing)**: for each seed, build the world, rewrite every passage via the local LLM, run the fact-preservation post-check, and store accepted rewrites in `artifacts/naturalized_passages.json`.
- **What the cache does**: it is exactly what `load_naturalized_loader(CACHE)` reads so training/eval can use naturalized prose with identical facts/gold. No cache = templated prose; full cache = the hardened data used to train toward the target. It runs offline once, then just gets loaded.

In simple words

Deterministic = the same input always gives the same output — no coin-flips between runs. Put in the same seed and you always get the exact same entities, names, numbers and passages; a different seed gives a different but equally repeatable world. Think of the seed like a **recipe-card number**: seed 42 always bakes the same cake, seed 99 always bakes a different, fixed one.
A **seed fixes the randomness**; it does **not** measure "how much". The generator has a fixed pseudo-random sequence, and the seed simply picks *which* starting point — so a seed selects *which* random world you get, not an amount of randomness. Same seed = same world (reproducible); one seed apart = a different, independent-looking world. The variety you cover is set by how many seeds you include, not by any single seed value: more seeds = more distinct worlds, because each is a different draw. A seed is just an ID that locks in one specific, reproducible world. A **range** tells the script *which* of those locked-in worlds to naturalize, because the cache stores one entry per seed:doc_id — a seed not in the cache simply keeps its original templated prose. A **complete cache** means every world used for training or eval has varied, natural prose with its facts unchanged. All of this runs once, offline and free via local Ollama, and is then just loaded at train time.

3. Verification (all green)

Oracle evaluation against the dev set (6/type):

Metric	Result
TASK SUCCESS	100.0%
final answer correct	100.0%
tool selection / args match oracle	100.0% / 100.0%
tool-necessity decision	100.0%
format strict / parseable	100.0% / 100.0%

Regression test suites — all passing:

- **test_pipeline.py** — ALL PASS (generation determinism, oracle 100%, toolset single-tool, SFT export, advantage ordering).
- **test_parser.py** — 0 failures (19 parser cases).
- **test_fix2.py** — ALL PASS (verifiers, efficiency, GRPO, cleanup).
- **test_curriculum_feedback.py** — ALL PASS (GRPO curriculum-feedback feature).
- **test_shortcut_filter.py** — 7/7 PASS. **test_naturalize.py** — 7/7 PASS. **verify_dataset.py** — ALL PASS (dataset end-to-end).

New test: test_shortcut_filter.py — 7/7 PASS

Test	Verifies
test_detection_not_vacuous	Filter flags at least one shortcut chain (not dead code).
test_not_overflagging	>= 50% of genuine chains are NOT flagged.
test_filter_excludes_rejected_shape	A flagged chain is excluded when filtering is on.
test_filter_observable_rejection_rate	Rejection counter non-zero with a sane rate.
test_dev_uses_dev_only_shapes	Dev set uses exactly the held-out shapes.
test_train_never_uses_dev_only_shapes	Train generation never emits a dev-only shape.
test_shapes_are_adjacent_and_disjoint	Dev shapes are not duplicated in train.

New test: test_naturalize.py — 7/7 PASS

Test	Verifies
test_facts_preserved_on_success	Good rewrite keeps every fact AND varies the text.
test_dropped_fact_is_rejected_and_falls_back	Dropped fact rejected; falls back after retries.
test_api_never_fabricates_facts	Only text/title touched; facts dict never mutated.
test_numfmt_matches_commas	Number formatting robust (1097000 == 9,800,000).
test_scoring_isolation	attrs / gold / oracle answers identical after naturalization.
test_loader_opt_in_wiring	build_world(text_loader=...) swaps prose only.
test_loader_none_when_missing	Missing/empty cache -> loader is None (opt-in).

Dataset end-to-end verification (all 5 families, naturalized path)

A dedicated `tests/verify_dataset.py` exercises the exact wired path that feeds `sft.jsonl` — generation + shortcut filter + route-pool holdout + naturalized loader — and evaluates the gold contract and oracle quality. Real measured results (200 train tasks, seeds 0–199):

Check	Result
All 5 families represented	no_tool 54 · musique_3hop 52 · musique_4hop 41 · unanswerab
Shortcut-filter rate	checked=150, rejected=24 → 16.0% (real leak, not dead code)
Gold invariant	180 answerable tasks all carry a value; 20 unanswerable correctly kind=none
Naturalization wired into build_world	28 docs rewritten across seeds 0–3 via text_loader
Dev set uses held-out dev shapes	dev generation pulls only the 3 held-out shapes (dev-only)
Oracle dev success (naturalized prose)	100.0% (30/30 answered) — full gen + loop path

Live naturalization (free, local, verified)

Ran the pipeline against a local **Ollama** server on 8 worlds (**304 passages**): **217 rewritten** (71.4%) with **0 fact-falls-back and 0 retries**, cache written to `artifacts/naturalized_passages_scaled.json`. An earlier focused run with the qwen2.5-coder:7b model rewrote 76/76 (100%) with zero fact-loss; the measurement above used the model actually installed on the local Ollama at run time (mistral-local:latest), which returns more conservative verbatim echoes — hence the lower rewrite share. Either way, **no passage ever ships broken prose** and every passage keeps its facts. No API key or credits required.

4. Primary files

- `atr/tasks/generator.py` — shortcut filter (`_is_shortcut_solvable`), train/dev route-pool split (`_ROUTES_TRAIN`/`_ROUTES_DEV_ONLY`), threaded `filter_shortcuts`/`route_pool`.
- `atr/tools/world.py` — `build_world(...)` opt-in `text_loader` for naturalized passages.
- `atr/agent/loop.py` — `LoopConfig.text_loader` so agent episodes see the same naturalized prose at tool-execution time.

- `atr/cli.py` — new `--cache` flag wired into `gen/eval/collect/ablate`; the loader flows through `_tasks_from()` → `generate/dev_set` → `build_world` → the agent loop (the exact `sft.jsonl` path).
- `atr/data/naturalize.py` (new) — fact-preserving passage rewrite + cache loader.
- `atr/data/naturalize_local.py` (new) — free, local runner that feeds the naturalize pipeline through Ollama's OpenAI-compatible endpoint (no key).
- `scripts/70_naturalize_passages.sh` (new) — offline naturalization to a seed-keyed cache.
- `tests/test_shortcut_filter.py`, `tests/test_naturalize.py`, `tests/verify_dataset.py` (new) — unit + end-to-end tests.

5. Models used

- **SFT / reinforcement (the agent model):** `Qwen/Qwen3-1.7B` ("only move to 4B once the 1.7B curve has flattened").
- **Part A naturalization LLM (live, free, local):** `qwen2.5-coder:7b` served by a local **Ollama** server (http://localhost:11434/v1), an OpenAI-compatible endpoint. No API key and no credits required — fully offline and on-machine. Run via `python -m atr.data.naturalize_local`.
- **Fallback option**: the same pipeline also works with any OpenAI-compatible provider (e.g. gpt-4.1-mini) if an API key is available; tests use a mock so no key is needed.

6. Next steps

- Naturalization is now **wired end-to-end into the training-data path** (`--cache` → generate/dev_set → build_world → loop) and verified with 100% oracle success on naturalized prose. Extend the seed/cache coverage to the full train/dev ranges via `scripts/70_naturalize_passages.sh` before the final SFT run.
- Reproduce the **20% → target** SFT training on these hardened data, then run GRPO, and report the held-out dev success (held-out shapes only) to confirm lifting above the baseline.
- A dead-code audit (pyflakes) now reports **zero issues** across the pipeline modules; the verified implementation and this report are committed to git on branch main.

Scope note: repository is a git repo on branch main with an unrelated pre-existing dirty file (requirements-colab.txt) deliberately left untouched. This summary reflects work verified locally on Windows/CPU.